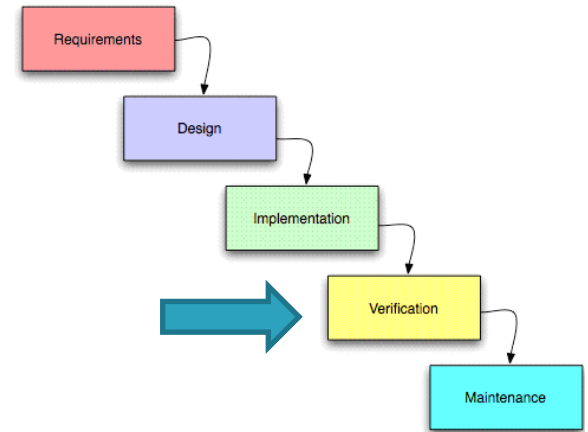


SOFTWARE TESTING: INTRODUZIONE

Ingegneria del software 2025/2026

AGENDA

- Terminologia:
 - Mediante l'utilizzo di una storiella ...
- Cosa è il **Software Testing**?
- Perché è difficile?
- Testing strutturale e funzionale
 - White-box vs. Black box
- Testing di unità, integrazione e sistema
- Testing di regressione
- **JUnit 5**
 - Framework per scrivere ed eseguire casi di test Java



INVESTIREOK

- Un **Trader/Broker** che lavora per la società *investireOK*
 - durante l'acquisto di **Futures “eurostoXX”**
 - usando la nuova funzionalità “*acquisto ciclico facile*” del sistema **EasyGuadagno 2.1**



FAILURE

- assiste a un:



- che poi si trasforma in **crash del sistema ...**

HELPDESK

- Il Trader/Broker contatta prontamente:



- la quale segue un **protocollo interno**

PROTOCOLLO INTERNO



L'assistenza:

1. Ascolta (ed annota) il problema
2. Spiega al Trader la **procedura di recovery** per far ripartire il sistema
 - Attenzione il **fault** che ha provocato il **failure** rimane nel sistema!
 - Conseguenza: il trader non può acquistare i futures ...
3. Attiva (apre) un **ticket** da inserire nel **bug tracking system** del sistema EasyGuadagno 2.1
 - Priorità massima! Affinché il **fault** venga risolto al più presto
4. Data la gravità viene avvertito il **project manager** e il **senior software developer**
 - Quest'ultimo (**Il guru**) conosce il sistema come le sue tasche ...

DUE SCENARI

- Il **guru** è in ufficio e si fa carico del problema
- Il **guru** è in vacanza ...



Spiaggia Lodo, Corsica

GURU IN UFFICIO (1)

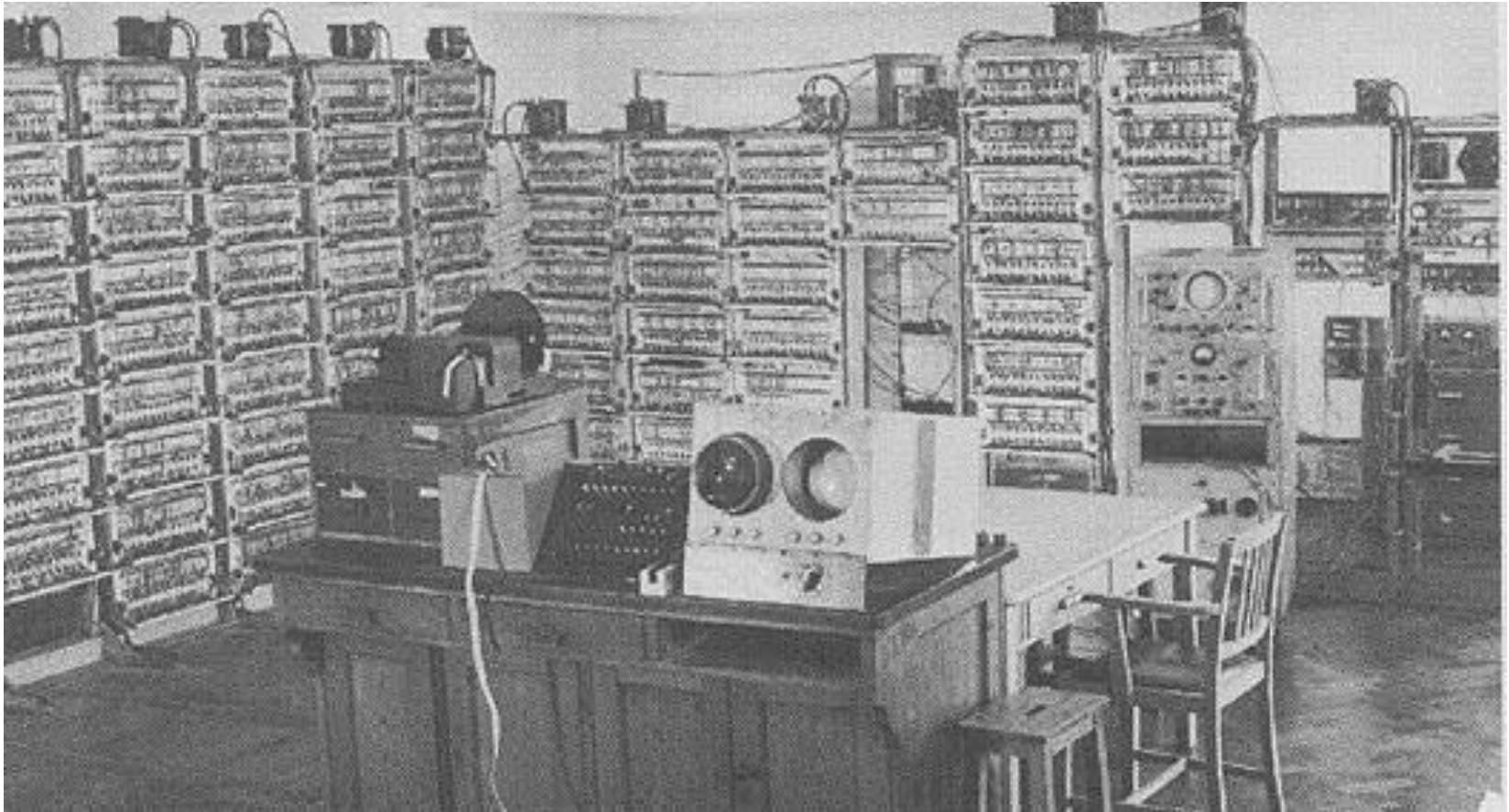
- Il **guru**:

1. Avvia una procedura di **debug**
 - A partire dalla descrizione del **failure**, usando gli stessi input del Trader cerca di riprodurre il **failure**
 - Viene individuato il file/componente in cui è presente il bug
 - Grazie all'esperienza del guru
 - Il **fault** (o **bug**) viene identificato
 - **fault localization**
 - Il bug viene rimosso con una **“patch”**
 - **bug fix**

First real “bug”



MARK II



ETIMOLOGIA

scienza che studia l'origine e la storia delle parole ...

ETIMOLOGIA

scienza che studia l'origine e la storia delle parole ...

9/9

0800 Antan started
1000 " stopped - antan ✓

		{ 1.2700	9.037 847 025
			9.037 846 995 correct
13" VC (032)	MP - MC	1.982147000 2.130476415	4.615925059 (-2)
(033)	PRO 2	2.130476415	
	correct	2.130676415	

Relays 6-2 in 033 failed special speed test
in relay " " test.

1100 Started Relays change
1525 Started Cosine Tap (Sine check)
Started Multiplier Adder Test.

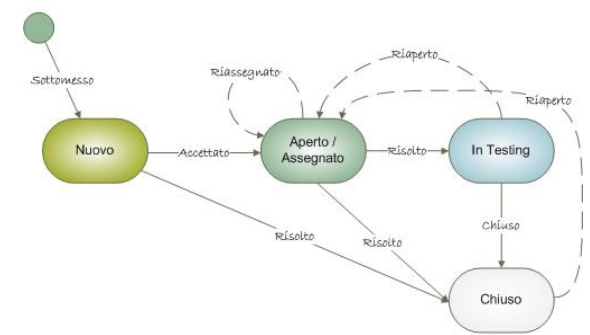
1545

Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.
 1630 and away started.
 1700 closed down.

1947: Tenente Hopper e il suo gruppo stavano cecando di capire come mai il Mark II non funzionava ...

GURU IN UFFICIO (2)



- Il **Guru** manda la **patch** al **senior software tester**
 - Il quale ri-esegue la batteria di casi di test (**testsuite**) per verificare che la patch non causi problemi ad altre funzionalità
 - Test di regressione**
- Setta il ticket a chiuso (nel **bug tracking system**)
- Aggiunge tutta la documentazione relativa all'intervento, compresa la patch, nel repository relativo alla **futura versione 2.2** di EasyGuadagno
- Avverte l'assistenza che è tutto ok
- L'assistenza poi avvertirà e rilascerà la **patch** al cliente

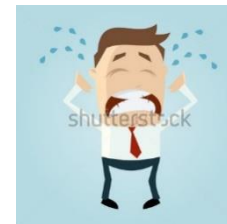
GURU IN VACANZA



La situazione si complica!

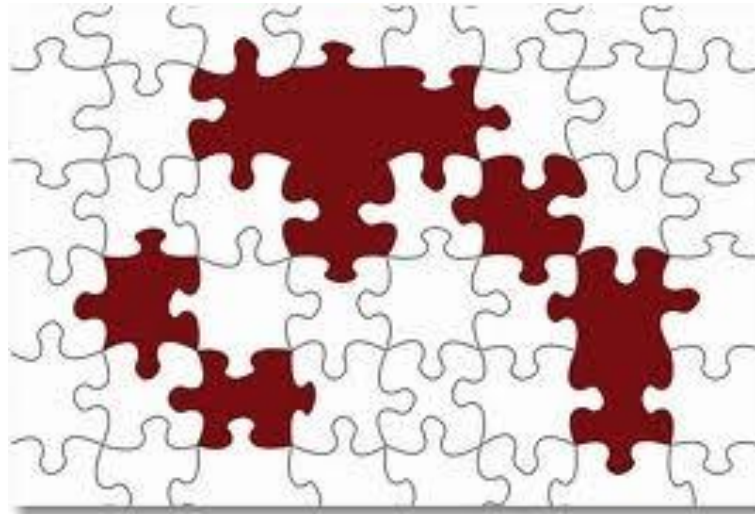
GURU IN VACANZA ...

- Il **Project manager** seleziona il primo sviluppatore che trova e lo nomina:
 - “**sostituto del Guru**”
- Il sostituto del Guru che non conosce il sistema:
 - Prova a riprodurre il failure
 - Ci riesce!
 - Prova ad avviare una **procedura di debug** ma il sistema è troppo complesso con decine e decine di files ...
 - Usa un **debugger**: prova a settare i **breakpoint**, esegue il programma ...
 - Ma non riesce a localizzare il file/componente che contiene il fault
 - E' disperato e telefona al Guru ...



VICE-GURU DISPERATO

- Si affida ad un **Fault localization tool** che analizza la descrizione testuale del failure e produce una lista di files che “**potrebbero**” contenere il fault
 - **Prototipi** basati su tecniche di *Natural Language Processing (NLP)* o *Large Language Model (LLM)*



- Grazie a questo aiuto il “vice-guru” riesce a trovare il file/componente che contiene il fault e procede con il **bug fix**

CAUSA DIRETTA DEL CRASH: “ERRORE”

- L'**errore** è stato commesso dallo sviluppatore “**Rossi**” nella seguente banale funzione ‘C’ di EasyGuadagno 2.1
 - che doveva raddoppiare un numero ($y = 2 * x$)
- L'errore può essere
 - un errore di battitura
 - un errore concettuale
 - non è chiaro il significato di “raddoppiare”
- **Fault:** alla linea 3
 - è stato usato “**x**” invece di “**2**”
- **Failure:** se eseguo reDouble(3) ottengo 9 e non 6 ...



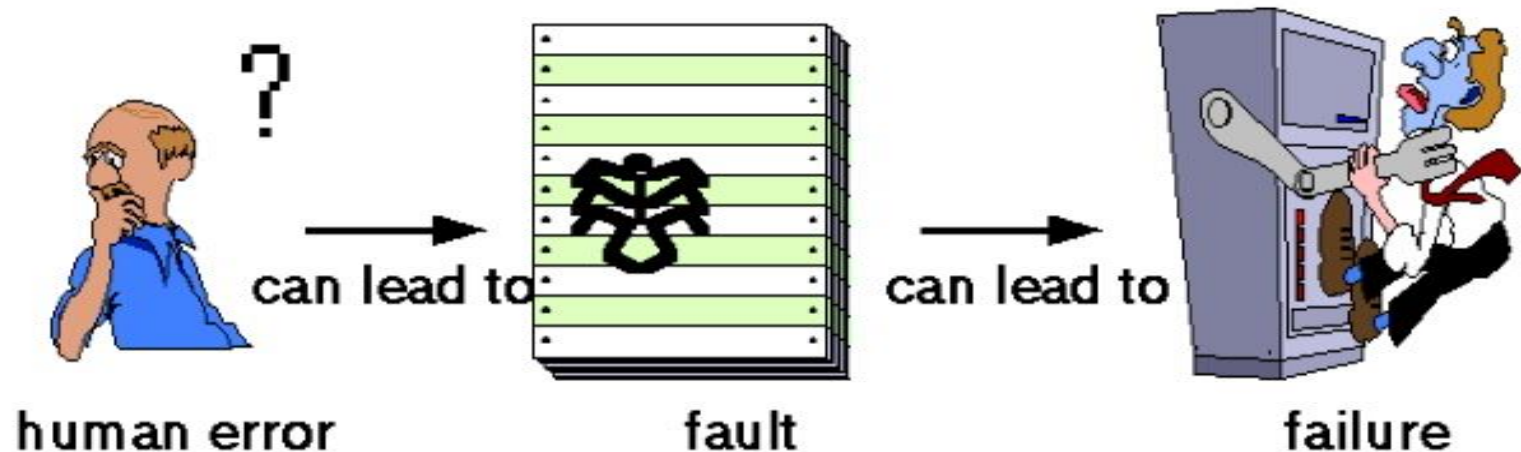
Codice

```
1  int reDouble (int x);
2  {  int y;
3    y := x * x;
4    return (y);
5  }
```

2

A blue arrow pointing from the number '2' below the code block to the variable 'x' in the expression 'y := x * x;' on line 3.

CATENA CAUSALE



Attenzione: un fault può sopravvivere all'interno di un programma a lungo. Almeno finchè non viene inserito un input che "sollecita" l'istruzione che lo contiene

CAUSA INDIRETTA DEL CRASH?

- È stato condotto **poco testing** e/o non sono state applicate in modo sistematico le **strategie/tecniche di testing opportune**

- Perché?

Capita spesso in pratica!



- **La fase di testing è:**

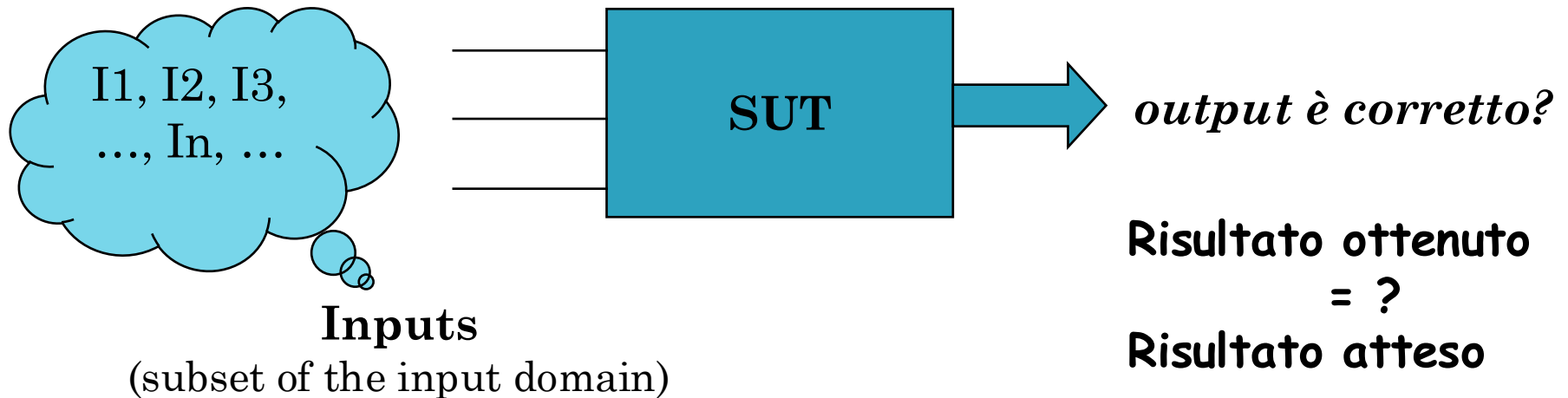
- molto difficile/complessa
 - selezionare gli input “giusti” per rivelare i failure
 - solitamente mancano le competenze necessarie
- costosa
- è l'ultima fase prima del rilascio per cui spesso non si ha tempo e risorse per condurla in modo adeguato



FINE DELLA STORIA INVESTIREOK ...

COSA È IL SOFTWARE TESTING?

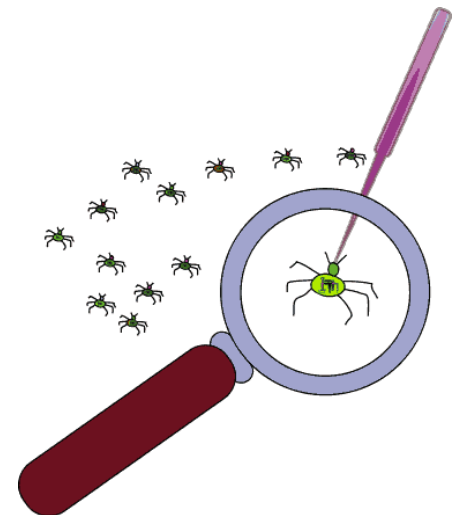
- E' una **procedura sistematica** che prevede l'**esecuzione** di un sistema software (SUT) con l'intento di trovare failure (e poi il fault associato)



Sut = system under test

DEBUGGING

- Software testing rivela i failure
 - ovvero un comportamento anomalo del SUT
- **Debugging**: il processo usato per trovare un bug/fault e rimuoverlo
- Due fasi:
 - **fault localization/location**
 - es. individuare il file che contiene il fault
individuare la linea che contiene il fault
 - **fault removal**



PERCHÈ IL TESTING È DIFFICILE (1)?

- Testing **esaustivo** è “**non realizzabile**” nei casi reali
 - Es:
 - Testare completamente un software che esegue la somma di 2 interi a 32 bit
 - Occorre testare il programma **per tutti gli input possibili**, ovvero tutte le possibili combinazioni
 - 2^{64} input = *18.446.744.073.709.551.616*
 - 1 run al millisec = **584.942.417 anni**

264

0000000000000000000000000000 + 0000000000000000000000000000 = 0000000000000000000000000000
 00000000000000000000000000001 + 0000000000000000000000000000 = 00000000000000000000000000001
 00000000000000000000000000001 + 00000000000000000000000000001 = 00000000000000000000000000010

[illegible]

PERCHÈ IL TESTING È DIFFICILE (2)?

- Il Testing è difficile perchè occorre selezionare **“buoni” input** nel dominio di tutti gli input possibili
 - “buoni” = con alta probabilità di trovare un fault ...



COME TROVARE BUONI INPUT?

- Due (macro) possibilità:
 - Affidarsi ad approcci white box o black box



**BLACK
BOX**

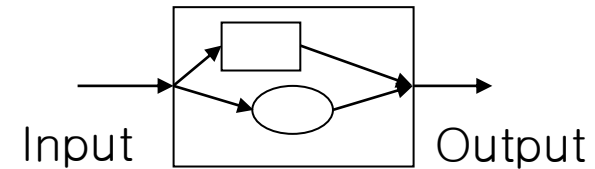
VS



**WHITE
BOX**

DUE “MACRO” CATEGORIE

- Il **white box testing** è basato su una conoscenza esplicita del SUT e della sua struttura
 - chiamato anche **structural testing**
 - *statement coverage testing* è un esempio
 - Lo scopo è quello di produrre abbastanza input per assicurare che ogni statement è eseguito almeno una volta
- Il **black box testing** non è basato su una conoscenza del SUT e della sua struttura
 - chiamato anche **functional testing**
 - di solito gli input sono generati a partire dai **requisiti/specifiche**



QUALE APPROCCIO È IL MIGLIORE?

- In assoluto non è possibile rispondere, dipende da caso a caso
 - **Troppe variabili:** SUT, approccio scelto, tester, ...
- In generale possiamo dire che sono complementari ...
- Anche se esiste uno studio che afferma:
 - **“Con tester esperti il testing funzionale rivela più fault del testing strutturale”**

“Comparing the effectiveness of software testing strategies”

V. Basili and R.W. Selby

IEEE Transactions on Software Engineering

Volume 13 , Issue 12 (December 1987)

Pages: 1278-1296

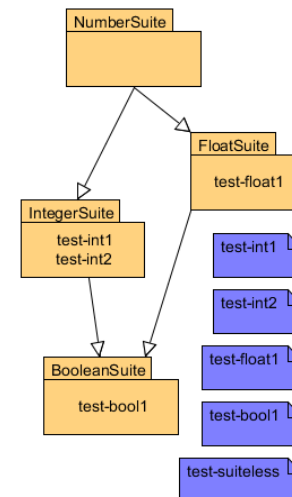
Year of Publication: **1987**

DEFINIZIONI DI TESTCASE E TESTSUITE

- Esistono diverse definizioni di testcase (**caso di test**)

IEEE Standard **Glossary** of Software Engineering Terminology

- A set of **inputs**, execution preconditions, and **expected outcomes** developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement [IEEE, Std 610.121990]
- Testsuite** = collezione di Testcases



ESEMPI (LA DEFINIZIONE VA ISTANZIATA)

Che cosa è “concretamente” un testcase dipende dal contesto:

Test case per desktop app “Sort”:

- Input: <“C”, 12, -29, 32, > C=Ordine crescente
- Output atteso: -29 12 32

input
+
output

Test case per Web Apps “Carrello”:

Input:

1. login alla pagina <http://www.abc.it/home.html>
2. vai sulla pagina “acquisti”
3. seleziona prodotto X
4. aggiungi il prodotto al carrello

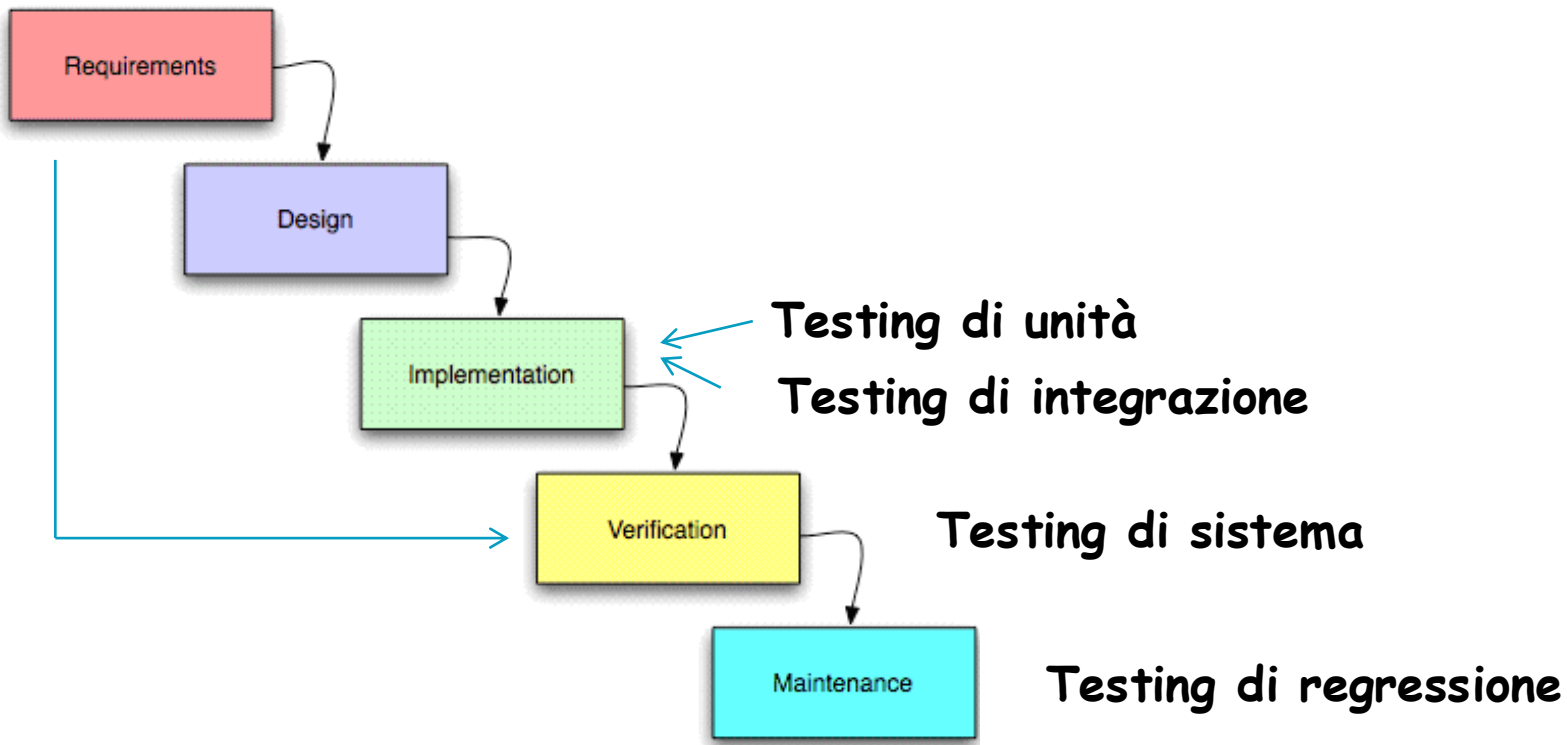
Output atteso:

Il prodotto X è stato aggiunto al carrello

sequence of steps
+
input
+
output

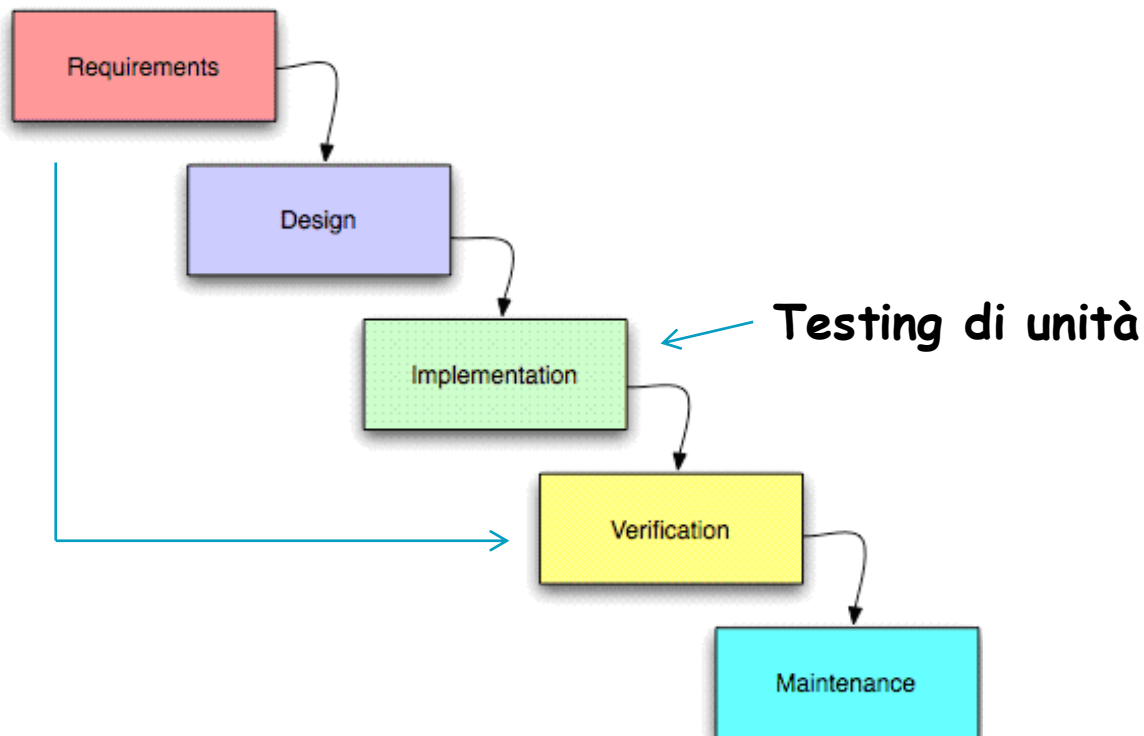
TIPOLOGIE DI TESTING (O LIVELLI)

- In relazione alla fase di sviluppo in cui viene condotto il testing:



TIPOLOGIE DI TESTING (1)

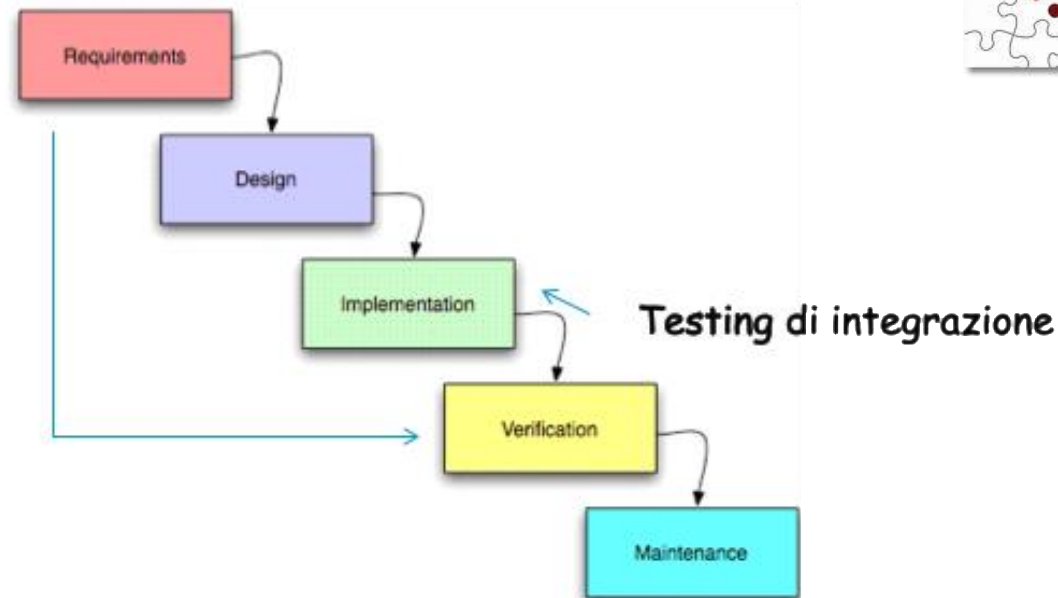
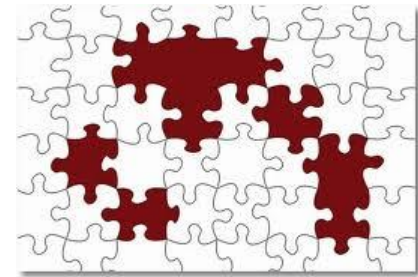
- **(implementazione) Testing di unità**
 - L'unità (funzione, procedura, classe, ...) viene testata dallo *sviluppatore* isolandola il più possibile dal resto
 - Concetto di stub / mock object (vedi TAP)
 - Di solito approccio **white-box**



TIPOLOGIE DI TESTING (1)

◦ (integrazione) Testing di integrazione

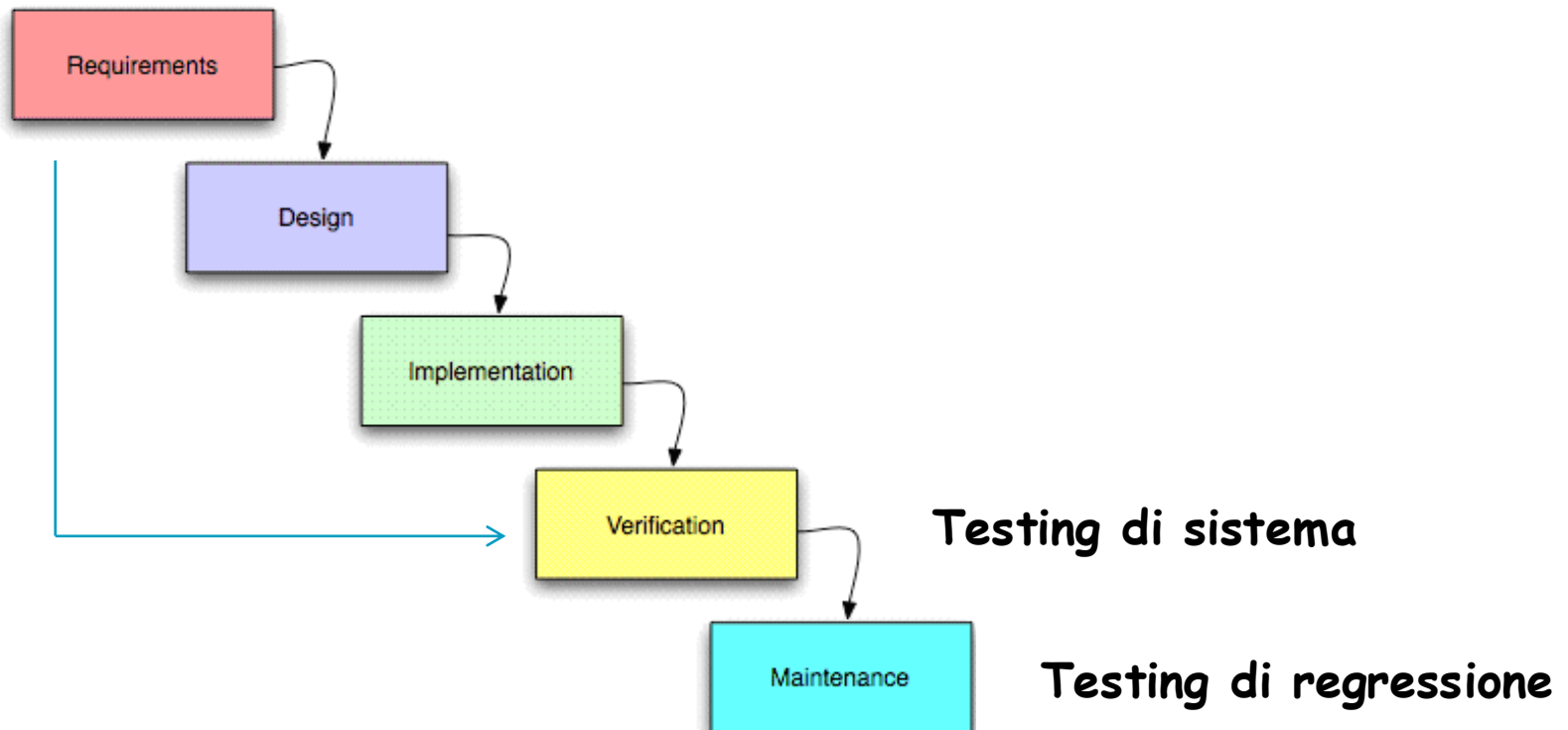
- I moduli sono “testati assieme”
- Viene testato come comunicano i moduli e quali info vengono scambiate
- **Interfacce** tra moduli di solito sono **fonti di errori**
 - Ordine sbagliato dei parametri
 - Precondizioni non rispettate
 - Protocollo di comunicazione non rispettato
 -



TIPOLOGIE DI TESTING (2)

○ (integrazione di sistema) Testing di sistema

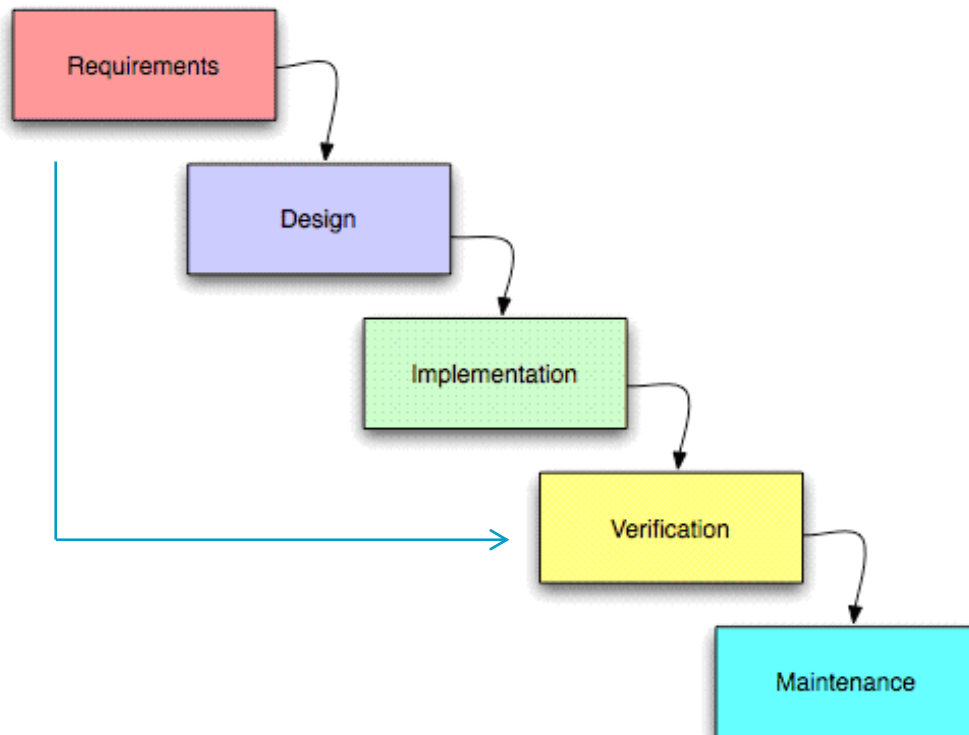
- Il “**Testing group**” verifica che il software soddisfa i requisiti e anche i bisogni dell’utente (**acceptance testing**)
- Di solito **black-box**
- Non solo aspetti di correttezza ma anche **performance**, **usabilità**, **security**, ...



TIPOLOGIE DI TESTING (2)

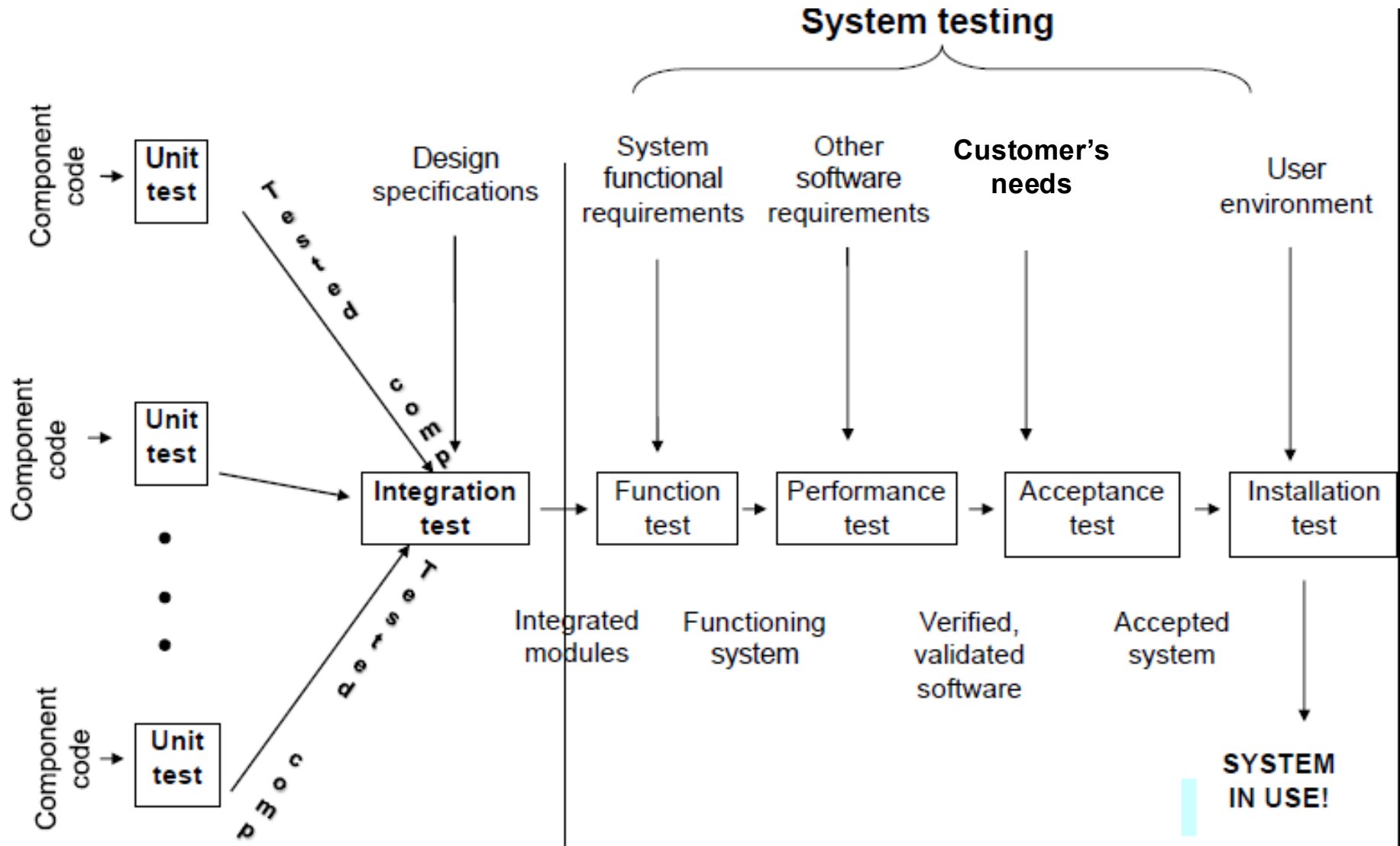
○ (manutenzione) Testing di regressione

- Tutte le volte che si effettua una modifica in un applicazione c'è il rischio di *side effect* in zone che apparentemente non sembrerebbero impattate dalla modifica
- Scopo del test di regressione è verificare che **non ci siano stati side effect (o regressioni)**



Testing di regressione

METTENDO TUTTO ASSIEME ...



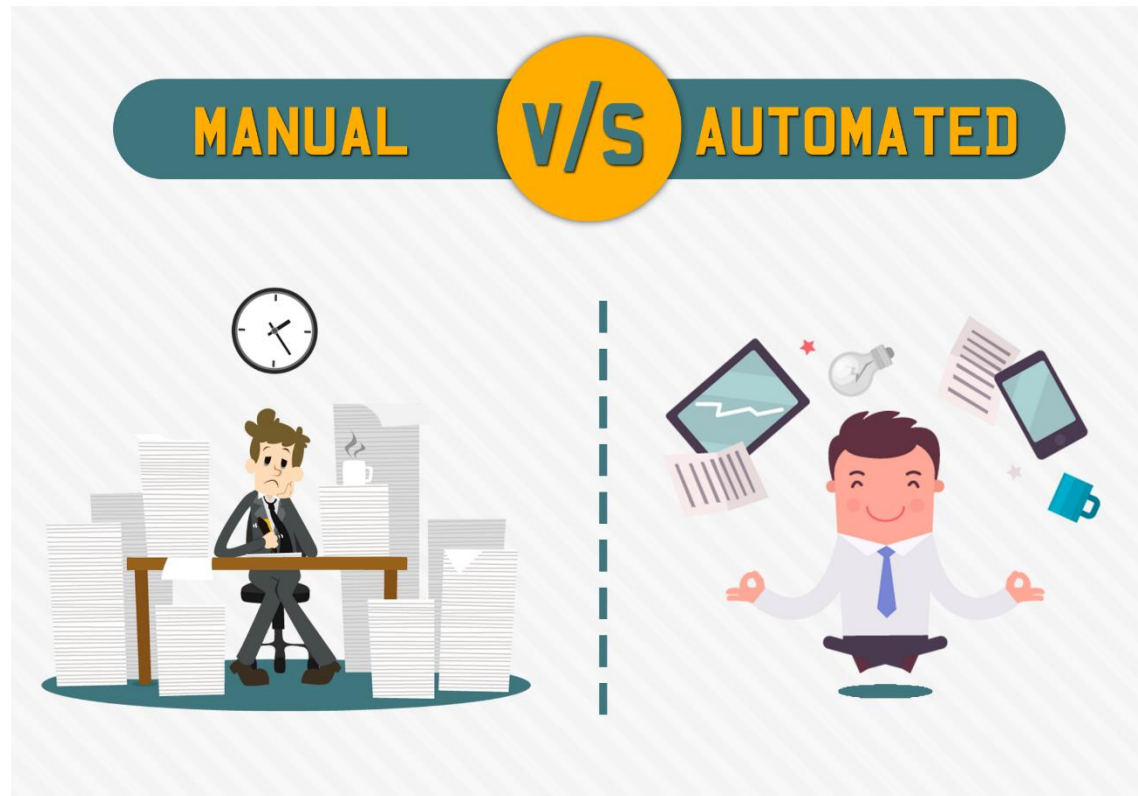


JUnit 5.0: INTRODUZIONE

<https://junit.org/junit5/>

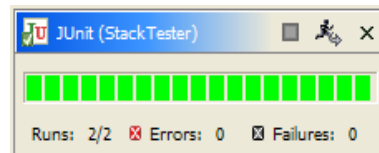
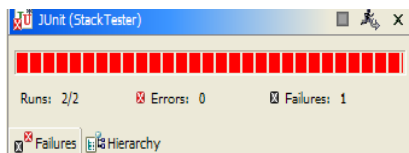
TESTING MANUALE VS. TEST AUTOMATIZZATO

- I tester scrivono test script (porzioni di codice) che sono eseguiti da un framework che li esegue e riporta il risultato (pass/fail)



COSA è JUnit?

- E' un **framework di testing** per programmi **Java**
 - *Unit testing*
 - Framework Java = insieme di classi e convenzioni per usarle
 - Largo utilizzo di annotazioni (@Test)
- Sviluppato da:
 - *Erich Gamma*
 - Anche Design Pattern e Eclipse ...
 - *Kent Beck*
 - Anche Extreme Programming e TDD ...
- Integrato in **IntelliJ** attraverso un plug-in grafico



COSA POSSIAMO FARE CON JUNIT?

1. Scrivere casi di test

- È codice Java!
- Test class
 - Test methods
 - “sequenza di chiamate a metodi + input + expected values”

2. Eseguire i casi di test e vedere il risultato

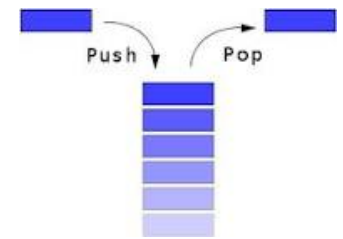
- **Pass** / **Fail**

3. Raggruppare i casi di test in testsuite

- **Buona pratica:** per ogni classe del progetto scrivere una corrispettiva classe di Test
 - Es. **Stack --> StackTest**

Versione corrente: JUnit 5 (5.5.2)
Disponibile a: junit.org

USARE IL 'MAIN' PER IL TESTING ...



- I **casi di test JUnit** “sostituiscono” l’uso del `main()` per testare la correttezza di una classe

```
public class Stack {  
    ...  
    public static void main (String[] args) {  
        Stack aStack = new Stack();  
        if (!aStack.isEmpty()) {  
            System.out.println ("Stack should be empty!");  
        }  
        aStack.push(10);  
        aStack.push(-4);  
        System.out.println("Last element:" + aStack.pop());  
        System.out.println("First element: " + aStack.pop());  
    }  
}
```

isEmpty()
pop()
push(...)

-4

10

Output atteso

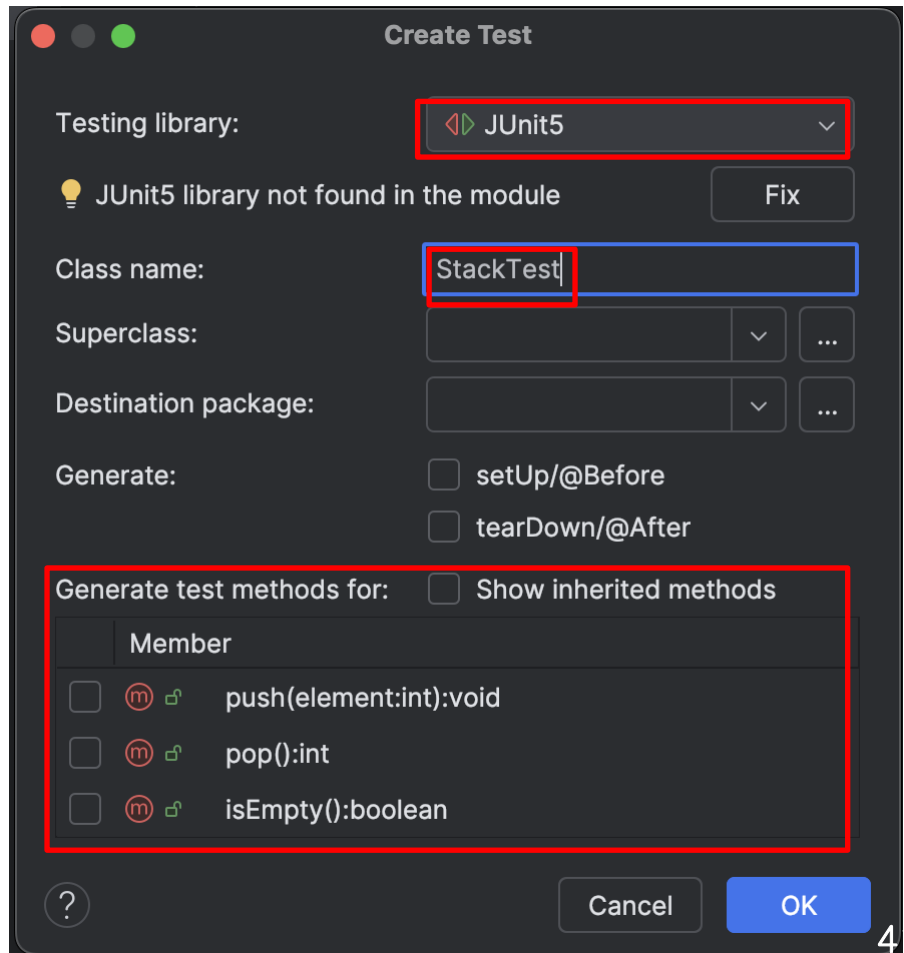
CON JUNIT

- Creiamo un nuovo progetto Eclipse e implementiamo una **classe Stack** da testare

Creiamo una classe contenitore di test case (**StackTest**) dal **wizard** “**Generate**” → “**Test Case**”



Volendo è possibile selezionare i metodi da testare



RISULTATO DEL WIZARD ...

```
public class Stack {  
    isEmpty()  
    pop()  
    push(...)  
}
```

Senza selezionare i metodi

import di classi ed annotazioni
del framework JUnit

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;
```

nome della classe di test

```
public class StackTest {
```

è un'annotazione per marcare i metodi che
si considerano di Test

```
@Test
```

```
    public void test() {
```

```
    }
```

```
}
```

ASERZIONI

- Rispetto all'esempio di prima in cui usavamo una **'print'** adesso i risultati attesi sono documentati con delle **asserzioni esplicite**, non con delle stampe che comunque richiedono dispendiose ispezioni dei risultati
- **Asserzione** = affermazione che può essere vera o falsa

```
assertEquals(-4, aStack.pop());
```


expected **actual**

- Se l'asserzione è:
 - vera: il test è andato a buon fine
 - In gergo si dice che il caso di test è **'passato'**
 - falsa: il test è fallito ed il codice testato non si comporta come atteso

PRIMO CASO DI TEST ...

```
public class Stack {  
    isEmpty()  
    pop()  
    push(...)  
}
```

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;  
  
public class StackTest {  
  
    @Test  
    public void testStack() {  
        Stack aStack = new Stack();  
        assertTrue(aStack.isEmpty());  
        aStack.push(10);  
        assertTrue(!aStack.isEmpty());  
        aStack.push(-4);  
        assertEquals(-4, aStack.pop());  
        assertEquals(10, aStack.pop());  
    }  
}
```

assertTrue

assertEquals

PRIMO CASO DI TEST ...

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

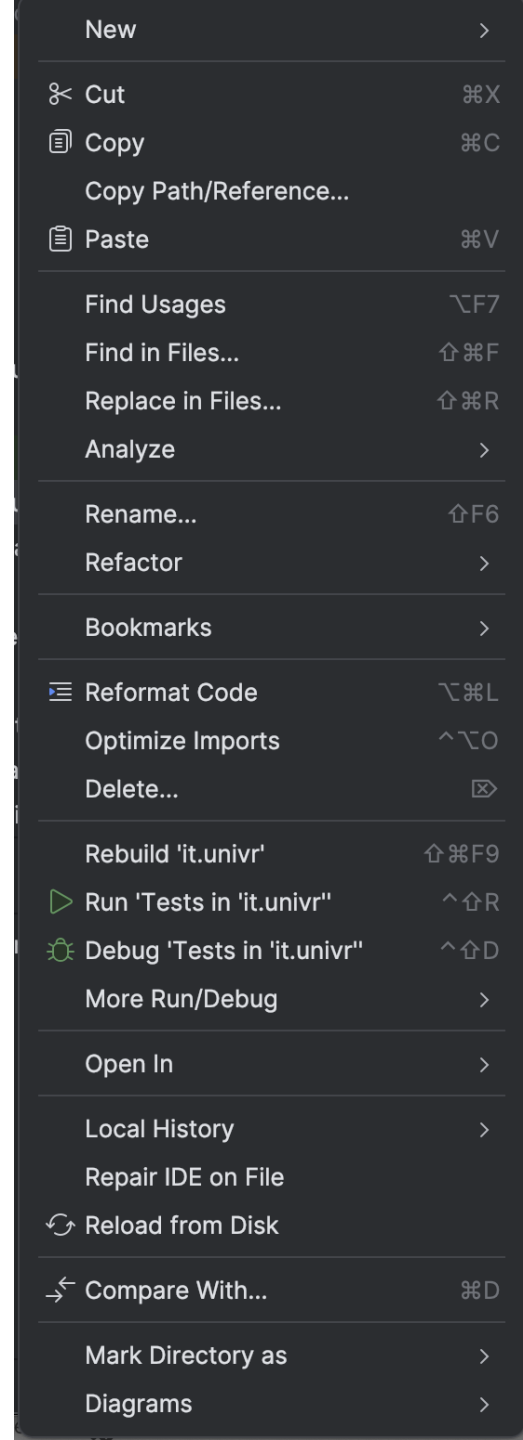
public class StackTest {

    @Test
    public void testStack() {
        Stack aStack = new Stack();
        assertTrue(aStack.isEmpty(), "lo stack deve essere vuoto");
        aStack.push(10);
        assertTrue(!aStack.isEmpty());
        aStack.push(-4);
        assertEquals(-4, aStack.pop());
        assertEquals(10, aStack.pop());
    }
}
```

Stampa sulla
Console il msg se
l'asserzione è falsa

RUN JUNIT TEST

- Seleziona Test class (o Test suite) da eseguire
- Run test in ...



TEST RUNNER (KO) ...



```
public class Stack {  
    isEmpty()  
    pop() 'ritorna sempre -1'  
    push(...)  
}
```

The screenshot shows an IDE's test runner interface. On the left, a sidebar lists test results: 'Test Results' (30 ms), 'StackTest' (30 ms), and 'testStack' (30 ms). The 'testStack' item is selected. The main area displays the test failure details for 'testStack'. It shows 'Expected :-4' and 'Actual :-1', with a link to 'Click to see difference'. Below this, the stack trace is visible, starting with 'org.opentest4j.AssertionFailedError' and listing several internal lines and Gradle worker threads.

Run StackTest x

Test Results 30 ms

StackTest 30 ms

testStack 30 ms

Tests failed 1 of 1 test – 30 ms

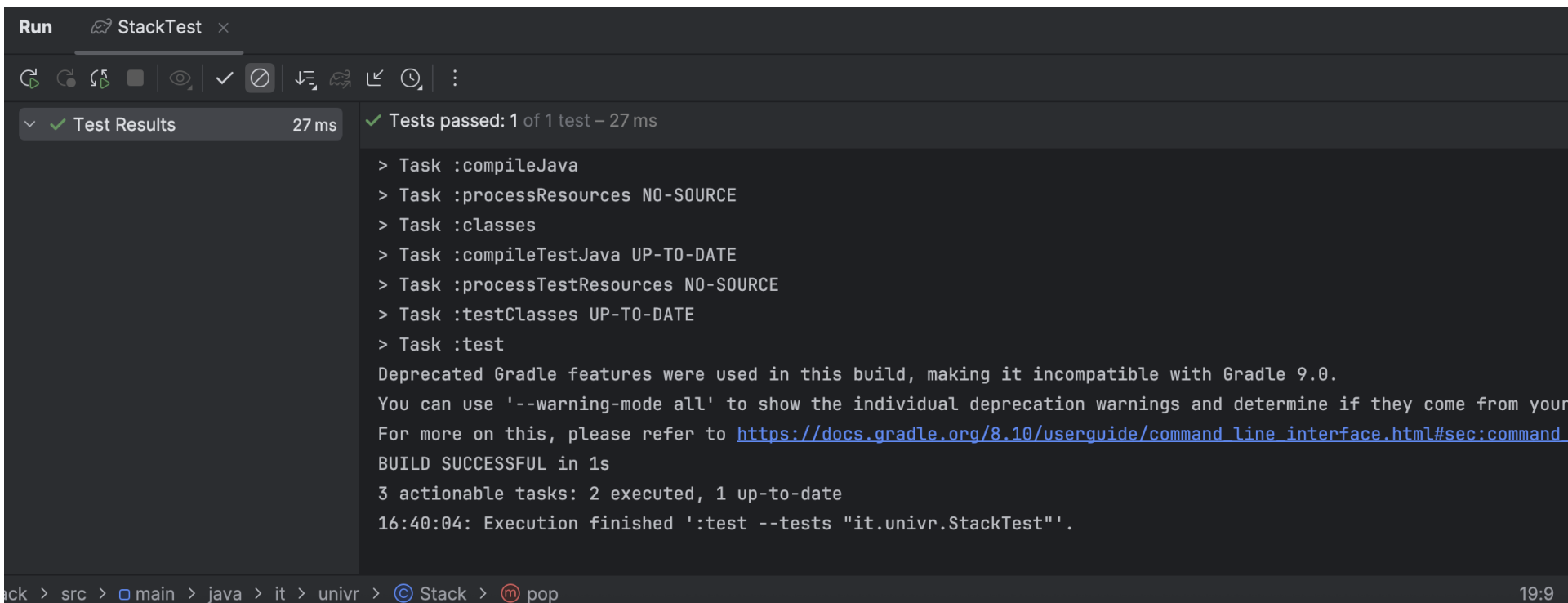
Expected :-4
Actual :-1
[Click to see difference](#)

> org.opentest4j.AssertionFailedError Create breakpoint : expected: <-4> but was: <-1> <5 internal lines>
> at it.univr.StackTest.testStack(StackTest.java:15) <29 internal lines>
> at java.base/java.util.ArrayList.forEach(ArrayList.java:1596) <9 internal lines>
> at java.base/java.util.ArrayList.forEach(ArrayList.java:1596) <30 internal lines>
> at jdk.proxy1/jdk.proxy1.\$Proxy4.stop(Unknown Source) <7 internal lines>
at worker.org.gradle.process.internal.worker.GradleWorkerMain.run(GradleWorkerMain.java:69)
at worker.org.gradle.process.internal.worker.GradleWorkerMain.main(GradleWorkerMain.java:74)

StackTest

TEST RUNNER (OK) ...

```
public class Stack {  
    isEmpty()  
    pop()  
    push(...)  
}
```



The screenshot shows the 'Run' window of an IDE with a tab for 'StackTest'. The toolbar includes icons for running, debugging, and other actions. The 'Test Results' section shows a green checkmark and '27 ms'. The main output area displays the following text:

```
✓ Tests passed: 1 of 1 test – 27 ms  
  
> Task :compileJava  
> Task :processResources NO-SOURCE  
> Task :classes  
> Task :compileTestJava UP-TO-DATE  
> Task :processTestResources NO-SOURCE  
> Task :testClasses UP-TO-DATE  
> Task :test  
  
Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.  
You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your  
For more on this, please refer to https://docs.gradle.org/8.10/userguide/command\_line\_interface.html#sec:command  
BUILD SUCCESSFUL in 1s  
3 actionable tasks: 2 executed, 1 up-to-date  
16:40:04: Execution finished ':test --tests "it.univr.StackTest"'.  
  
ack > src > main > java > it > univr > © Stack > (m) pop
```

FAILURE VS. ERROR

- Failure: un'asserzione nel Test method non soddisfatta

- `assertEquals( -3, aStack.pop() );`

`expected <-3> but was <-4>`

- Error: errore a run-time del codice che stiamo testando

- `assertEquals( -3, saldo.getSaldo() );`

`java.lang.ArrayIndexOutOfBoundsException ...`

COSA 'NON VA' IN QUESTO CASO DI TEST?

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
```

```
public class StackTest {
```

```
    @Test
```

```
    public void testStack() {
```

```
        Stack aStack = new Stack();
```

```
        assertTrue(aStack.isEmpty());
```

```
        aStack.push(10);
```

```
        assertTrue(!aStack.isEmpty());
```

```
        aStack.push(-4);
```

```
        assertEquals(-4, aStack.pop());
```

```
        assertEquals(10, aStack.pop());
```

```
    }
```

```
}
```

Non è coeso!

UNA COSA ALLA VOLTA ...

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;
```

```
public class StackTest {
```

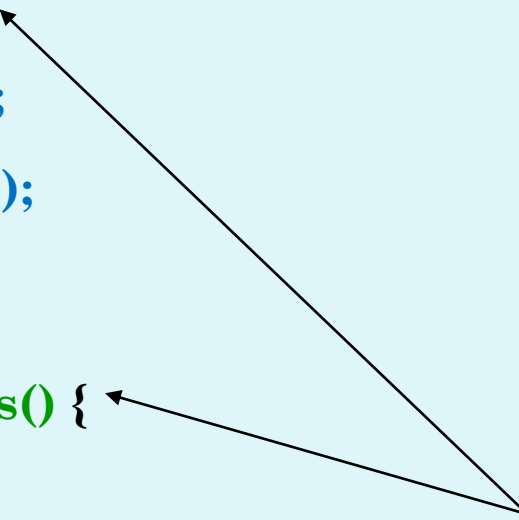
```
    @Test
```

```
    public void testStackEmpty() {  
        Stack aStack = new Stack();  
        assertTrue(aStack.isEmpty());  
        aStack.push(10);  
        assertTrue(!aStack.isEmpty());  
    }
```

```
    @Test
```

```
    public void testStackOperations() {  
        Stack aStack = new Stack();  
        aStack.push(10);  
        aStack.push(-4);  
        assertEquals(-4, aStack.pop());  
        assertEquals(10, aStack.pop());  
    }  
}
```

Occorre "separation of concerns"



UNA COSA ALLA VOLTA ...

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
```

```
public class StackTest {
```

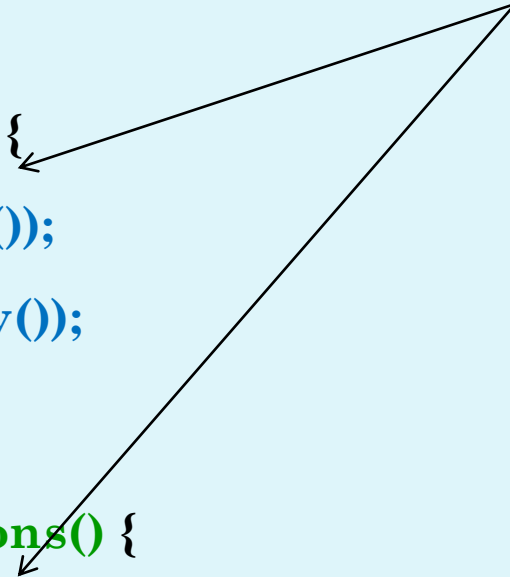
Inizializzazione? Cloni ...

```
@Test
```

```
public void testStackEmpty() {
    Stack aStack = new Stack();
    assertTrue(aStack.isEmpty());
    aStack.push(10);
    assertTrue(!aStack.isEmpty());
}
```

```
@Test
```

```
public void testStackOperations() {
    Stack aStack = new Stack();
    aStack.push(10);
    aStack.push(-4);
    assertEquals(-4, aStack.pop());
    assertEquals(10, aStack.pop());
}
}
```



BEFOREEACH E AFTEREACH

- **@BeforeEach** setUp() **inizializza gli oggetti da testare**
 - metodo chiamato prima di ogni test method
 - lo stato è sempre lo stesso per ogni test method
- **@AfterEach** tearDown() **rilascia gli oggetti**
 - metodo chiamato dopo ogni test method

```
ShoppingCart cart;  
Book book1, book2;  
  
@BeforeEach  
protected void setUp() {  
    cart = new ShoppingCart();  
    book1 = new Book("JUnit", 29.95);  
    book2 = new Book("XP", 18.25);  
    cart.addItem(book1);  
    cart.addItem(book2);  
}
```

```
public class StackTest  
    extends TestCase {  
    @BeforeEach setUp() {...}  
    @AfterEach tearDown {...}  
    @Test testIsEmpty() {...}  
    @Test testPop() {...}  
    @Test testPush(...) {...}  
}
```



```
setUp()  
testIsEmpty()  
tearDown()
```

```
setUp()  
testPop()  
tearDown()
```

```
setUp()  
testPush()  
tearDown()
```

ESEMPIO: @BEFOREEACH

```
public class StackTest {  
    Stack aStack;  
    @BeforeEach  
    public void setUp() {aStack = new Stack();}  
    @Test  
    public void testStackEmpty() {  
        Stack aStack = new Stack();  
        assertTrue(aStack.isEmpty());  
        aStack.push(10);  
        assertTrue(!aStack.isEmpty());  
    }  
    @Test  
    public void testStackOperations() {  
        Stack aStack = new Stack();  
        aStack.push(10);  
        aStack.push(-4);  
        assertEquals(-4, aStack.pop());  
        assertEquals(10, aStack.pop());  
    }  
}
```

COSA ACCADE QUANDO CI SONO PIÙ ASSERZIONI?

```
public void testStackOperations() {  
    Stack aStack = new Stack();  
    aStack.push(10);  
    aStack.push(-4);  
     assertEquals(-4, aStack.pop(), "pop() error", );  
    assertEquals(10, aStack.pop());  
}
```

- Se un **asserzione è falsa**:
 1. test fallisce immediatamente (**red** bar)
 2. la parte restante del metodo viene “saltata”
 3. il messaggio (se presente) è stampato **pop() error**
- Se un **asserzione è vera**:
 - esecuzione continua normalmente
- Se tutte le asserzioni nel Test method sono vere allora il test passa (**green** bar)

ASSERT*() E FAIL()

`assertEquals( -3, aStack.pop() );`

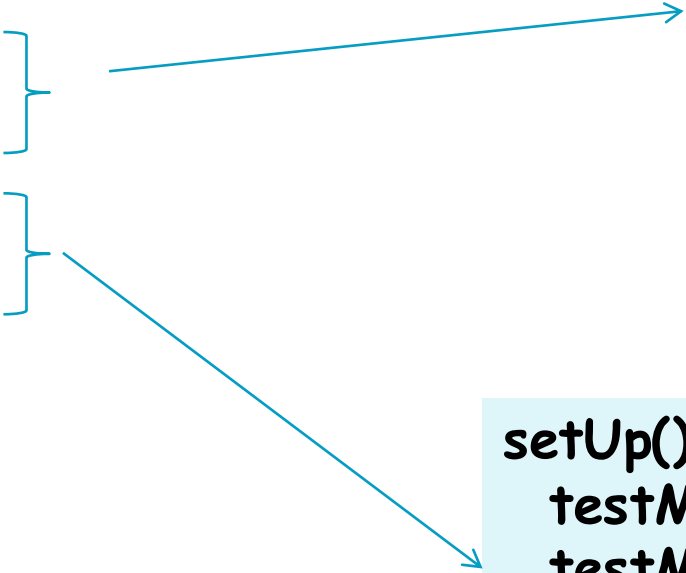
- Per una condizione booleana
 - `assertTrue(boolean condition, “message for fail”);`
- Per objects, int, long, boolean, strings, byte ...
 - `assertEquals(expected_value, actual, “message for fail”);`
 - per objects verrà usato il metodo equals definito
 - Il metodo equals per gli array non confronta il contenuto ma il riferimento all'array
- Per confrontare array usare `assertArrayEquals(expectet_value, actual);`
- Per float e double values
 - `assertEquals (expected, actual, error);`
- Controlla se expected e actual fanno riferimento allo stesso oggetto
 - `assertSame(expected_value, actual);`
 - `assertNotSame(expected_value, actual);`
- Fa fallire il test immediatamente
 - `fail (“message for fail” );`

Tolleranza

JUNIT 5 FORNISCe

- **Inizializzazione flessibile:**

- @BeforeEach }
◦ @AfterEach }
◦ @BeforeAll }
◦ @AfterAll }



```
setUp()  
  testMethod1()  
tearDown()  
  
setUp()  
  testMethod2()  
tearDown()  
  
setUp()  
  testMethod3()  
tearDown()
```

```
setUp()  
  testMethod1()  
  testMethod2()  
  testMethod3()  
tearDown()
```

JUNIT 5 FORNISCHE ANCHE

○ Test parametrici:

```
@ParameterizedTest  
@ValueSource(strings = { "Hello", "JUnit" })  
void testWithValueSource(String word) {  
    assertNotNull(word);  
}
```

▼	✓ Test Results	26ms
▼	✓ HelloParams	26ms
▼	✓ testWithValueSource(String)	26ms
	✓ [1] Hello	25ms
	✓ [2] JUnit	1ms

PROGRAMMA DEL CORSO (TEORIA)

- Processo di sviluppo software
- Ingegneria dei Requisiti
- Progettazione del Software
 - Progettazione architetturale
 - Progettazione delle component
- Unified Modeling Language
 - Class Diagram
 - Sequence Diagram
 - Sequence Diagram
 - State Machine Diagram
 - Activity Diagram
- Software Design Patterns
- Evoluzione del Software
- Testing del Software

ESAME DEL CORSO

- Progetto assegnato durante il corso
 - Gruppo da due a tre partecipanti
- Sbarramento per accedere all'orale
 - Domande chiuse a scelta multipla in aula informatica
- Esame orale
 - Discussione sul progetto
 - Domande di teoria